Journal of Scientific Computing (2026) 108:24
https://doi.org/10.1007/s10915-026-03337-1

Parallel-in-Time Solution of Allen-Cahn Equations by Integrating Operator Learning into the Parareal Method

Yuwei Geng[1] · Junqi Yin[2] · Eric C. Cyr[3] · Guannan Zhang[4] · Lili Ju[1]

Received: 29 September 2025 / Revised: 4 May 2026 / Accepted: 18 May 2026 /
Published online: 2 June 2026
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract

While recent advances in deep learning have shown promising efficiency gains in solving time-dependent partial differential equations (PDEs), matching the accuracy of conventional numerical solvers still remains a challenge. One strategy to improve the accuracy of deep learning-based solutions for time-dependent PDEs is to use the learned model as the coarse propagator in the Parareal method and a traditional numerical method as the fine solver. However, successful integration of deep learning into the Parareal method requires consistency between the coarse and fine solvers, particularly for PDEs exhibiting rapid changes such as sharp transitions. To ensure this consistency, we propose using convolutional neural networks (CNNs) to learn the fully discrete time-stepping operator defined by the same numerical scheme employed as the fine solver. We demonstrate the effectiveness of the proposed method in solving the classical and mass-conservative Allen–Cahn (AC) equations. Through iterative updates in the Parareal algorithm, our approach achieves a significant computational speedup compared to traditional fine solvers while converging to high-accuracy solutions. Our results highlight that the proposed hybrid Parareal algorithm effectively accelerates simulations, particularly when implemented on multiple GPUs, and converges to the desired accuracy in only a few iterations. Another advantage of our method is that the CNN

✉ Lili Ju
ju@math.sc.edu

Yuwei Geng
ygeng@email.sc.edu

Junqi Yin
yinj@ornl.gov

Eric C. Cyr
eccyr@sandia.gov

Guannan Zhang
zhangg@ornl.gov

[1] Department of Mathematics, University of South Carolina, Columbia, SC 29208, USA

[2] National Center for Computational Sciences, Oak Ridge National Laboratory, Oak Ridge, TN 37831, USA

[3] Computational Mathematics Department, Sandia National Laboratories, Albuquerque, NM 87123, USA

[4] Computer Science and Mathematics Division, Oak Ridge National Laboratory, Oak Ridge, TN 37831, USA

model is trained on trajectory-based data generated from random initial conditions, such that the trained model can be used to solve the AC equations with various initial conditions without retraining. This work demonstrates the potential of integrating neural network methods into parallel-in-time frameworks for efficient and accurate simulations of time-dependent PDEs.

Keywords Allen-Cahn equations · Convolutional neural network · Parareal algorithm · GPU acceleration · Parallel computing

Mathematics Subject Classification 65M12 · 68T07 · 65Y05

1 Introduction

The Allen-Cahn (AC) equation was originally proposed in [2] as a phenomenological model to describe antiphase domain coarsening in binary alloys. Formulated as a time-dependent PDE, it defines an initial value problem where the temporal evolution of an order parameter captures phase transitions. Beyond its original context, the AC equation has been widely applied in modeling dynamic processes across various scientific and engineering domains, including crystal growth [33], image processing [3], and biological pattern formation [29]. Mathematically, analytical solutions to the AC equation are very challenging or even impossible to find. Hence, effective and accurate numerical methods for computing their approximate solutions become highly necessary in order to simulate and understand their dynamics. Typically, those are solved through numerical integration over time steps using, e.g., fully implicit schemes [9, 34], stabilized semi-implicit schemes [30, 31], or exponential integrator-based methods [5–7]. However, solving the AC equation requires very fine spatial and temporal discretizations, with mesh sizes smaller than the interface parameter and time steps restricted by stability conditions, making direct simulation costly. Although implicit methods relax time step restrictions, they still involve solving large systems, so simulations become particularly expensive when having sharp interfaces between phases. Recently, deep learning has provided a promising alternative in solving the AC equation, e.g., as in [11], to significantly improve computational efficiency. Nevertheless, deep-learning-based solutions of the AC equations often lack sufficient accuracy, as they struggle to capture the dynamics of sharp interface structures.

The overarching goal of this work is to integrate deep-learning-based AC solvers with traditional numerical solvers in a parallel-in-time framework, thereby accelerating the solution of both classical and mass-conservative AC equations while maintaining satisfactory accuracy. The Parareal method [20] is a parallel-in-time algorithm designed to accelerate the solution of time-dependent problems by using parallel computing resources. It has been extensively studied for time-dependent PDEs; see, for example, the recent comprehensive overview in [10], which provides a systematic treatment of Parareal and related parallel-in-time (PinT) algorithms for parabolic and hyperbolic problems. The Parareal algorithm iteratively combines two propagators: a coarse propagator that is computationally efficient and provides an approximate solution, and a fine propagator that is more accurate but computationally expensive. Each iteration uses corrections from the fine solver to refine the coarse prediction, gradually converging to a high-fidelity solution.

There are two emerging directions linking parallel-in-time integration with machine learning. One direction focuses on applying parallel-in-time principles to accelerate the training

of deep learning models. A multigrid reduction-in-time approach [8] is used to enable layer-parallel training of deep residual networks [12] by interpreting layers as a time domain and applying parallel-in-time techniques for efficient parameter updates. Extensions of this idea in [16] have demonstrated additional gains on multi-GPU platforms. Another strategy [24] aims to improve the training efficiency of physics-informed neural networks, especially for long time horizons. By breaking the time domain into shorter segments and applying Parareal to connect them, the training becomes more scalable. Other work includes adapting neural ordinary differential equations (ODEs) to a parallel-in-time framework to improve residual network training in [22], and proposing a Parareal-inspired neural network architecture for efficient parallel training of deep models across multiple GPUs in [18].

Another direction of research focuses on improving the performance of parallel-in-time algorithms using machine learning. For example, neural networks have been explored as coarse propagators in the Parareal algorithm in a series of efforts [1, 15, 26, 36]. Despite the initial success of combining Parareal with deep learning for solving PDEs, there are significant challenges in extending this approach to classical and mass-conservative AC equations. First, it is difficult to ensure consistency between the deep-learning model used as the coarse propagator and the traditional numerical solver used as the fine propagator, particularly for the AC equation, which has dynamically evolving sharp interfaces. Failing to ensure such consistency will require more global iterations in the Parareal algorithm, ultimately diminishing the computational benefits of using Parareal. Second, the deep-learning model should generalize across a wide range of initial conditions beyond those used in training. Otherwise, the need to re-train the model for each new initial condition would offset the computational benefits of using Parareal due to the added training cost.

To address these challenges, we propose training two neural network models [11], i.e., an Allen-Cahn Neural Network (ACNN) for the classical AC equation and a mass-conservative Allen-Cahn Neural Network (mACNN) for the mass-conservative AC equation, and integrating them into the Parareal framework to accelerate time-to-solution for both types of AC equations. The ACNN and mACNN models learn from fully discrete time-stepping operators for initial value problems targeting the classical and mass-conservative AC equations. As operator-learning approaches, they generalize across different initial conditions, therefore, do not require retraining when the input data changes. Since the models are trained using the fully discrete time-stepping scheme, they naturally remain consistent with the fine solver used in Parareal. Additionally, the neural networks can be executed with high computational efficiency, which enables simulations on sufficiently fine temporal meshes to ensure stability while only requiring coarse-mesh snapshots for Parareal iterations. Moreover, the convolutional neural network architecture enables them to capture sharp interfacial structures that are often difficult for other machine-learning-based solvers. We evaluated the proposed framework on both the classical and mass conservative AC equations and demonstrated its scalability across multiple compute nodes, achieving performance on up to 128 GPUs.

The rest of the paper is organized as follows. In Section 2, we introduce the numerical scheme for AC equations and review the ACNN and mACNN models. Section 3 discusses the Parareal algorithm and explains how it integrates with our deep-learning models as coarse propagators. In Section 4, we present comparisons in terms of accuracy and speedup to demonstrate the effectiveness of the Parareal algorithm. Finally, we conclude the paper with a brief discussion of future directions.

2 Problem Setting

We introduce the classical AC and mass-conservative AC (mAC) models, followed by the numerical schemes used for both equations. We then describe the network architectures for ACNN and mACNN. Let $\Omega \subset \mathbb{R}^d, d = 2, 3$ be a bounded domain. The classical AC equation is given by

$$\partial_t u = \epsilon^2 \Delta u + f(u) \tag{1}$$

equipped with periodic boundary conditions and a suitable initial condition $u(0, \boldsymbol{x}) = u_0(\boldsymbol{x})$, ϵ is the interfacial parameter, $u(\boldsymbol{x}, t)$ is the unknown scalar function and $f = -F'$, where $F(u)$ is a nonlinear potential function. The commonly used double-well potential function is adopted for this work:

$$F(u) = \frac{1}{4}(u^2 - 1)^2, \quad f(u) = u - u^3. \tag{2}$$

Based on the maximum bound principle (MBP) [7, 19], the solution of the classical AC equation (1) always remains within the interval $[-1, 1]$.

The classical AC equation does not preserve the total mass, defined as

$$M(t) = \int_\Omega u(\boldsymbol{x}, t) d\boldsymbol{x}. \tag{3}$$

To overcome this limitation, several mass *conservative* variants of the AC equation have been developed. A prominent example, proposed and analyzed in [28], introduces a nonlocal modification of the classical formulation:

$$\partial_t u = \epsilon^2 \Delta u + f(u) - \frac{1}{|\Omega|} \int_\Omega f(u(\boldsymbol{y}, t)) d\boldsymbol{y}, \quad \boldsymbol{x} \in \Omega, t > 0, \tag{4}$$

where the term $\frac{1}{|\Omega|} \int_\Omega f(u(\boldsymbol{y}, t)) d\boldsymbol{y}$ acts as a Lagrange multiplier to ensure mass conservation. This guarantees $M(t) = M(0)$ for all $t \geq 0$. Both the classical AC equation (1) and its conservative counterpart (4) are *autonomous* systems under periodic or homogeneous boundary conditions. The MBP principle implies that the solutions of the conservative AC equation (4) remain within the range $[-\frac{2}{3}\sqrt{3}, \frac{2}{3}\sqrt{3}]$. From a variational point of view, the classical AC equation is a L_2 gradient flow associated with the energy functional:

$$E(u) = \int_\Omega \left(\frac{\epsilon^2}{2} |\nabla u(\boldsymbol{x}, t)|^2 + F(u(\boldsymbol{x}, t)) \right) d\boldsymbol{x}. \tag{5}$$

The conservative AC equation (4), while enforcing mass conservation, also retains an energy dissipation property with respect to the same functional (5).

When solving the AC equation with traditional numerical schemes, spatial discretizations require the computational mesh size to be smaller than the interfacial parameter ϵ to accurately resolve the transition layers and capture the correct dynamics. In recent years, neural network models have gained increasing popularity for solving PDEs, owing to their flexibility and ability to approximate complex nonlinear dynamics. However, despite these advances, several notable challenges remain. First, many neural network models, such as standard PINNs [27], require retraining whenever the initial condition changes. Second, the presence of nonlocal terms in the mAC equation, particularly in higher dimensions, poses significant difficulties for conventional neural architectures. Third, capturing sharp interfacial structures remains a persistent challenge [17, 35], as neural networks often struggle with irregular or non-smooth solutions. Although several modified models have been proposed to address the individual

aspects of these difficulties [4, 14, 21, 23, 32, 38], no single framework has successfully resolved all of them simultaneously.

3 Operator-Learning-Enabled Parareal Algorithm for Allen-Cahn Equations

This section is organized as follows. We begin by presenting the fully discrete schemes introduced in Section 3.1. Next, Section 3.2 introduces the neural network model, including its architecture, loss function, and training procedure. Finally, in Section 3.3, we employ ACNN/mACNN as a coarse propagator within the Parareal framework.

3.1 The Fully-Discrete Schemes for the Allen-Cahn Equations

For notational simplicity, we present our methodology in a 2D setting, while the numerical experiments in Section 4 include both 2D and 3D cases. Specifically, we consider a two-dimensional square domain $\Omega = [0, L]^2$ and assume periodic boundary conditions for the solution u. Note that the numerical scheme is used not only as the fine solver but also as the basis for defining the loss function used to train the neural network model. Let us uniformly partition the domain Ω and obtain the set of grid points $\{(x_i, y_j)\}_{i,j=0}^{N}$ where $N > 0$ is an integer, $x_i = ih$ and $y_j = jh$ with $h = L/N$. Then we choose a uniform time step size $\Delta t > 0$ and set $t_n = n\Delta t$ for $n \geq 0$. Correspondingly, we set $U_0^{i,j} = u_0(x_i, y_j)$ and assume $U_n^{i,j} \approx u(x_i, y_j, t_n)$. The periodic boundary condition implies $U_n^{0,j} = U_n^{N,j}$ and $U_n^{j,0} = U_n^{j,N}$ for $j = 0, 1, \cdots N$, so the set of unknowns can be represented by $U_n = \{U_n^{i,j}\}_{i,j=1}^{N}$.

We adopt the second-order Crank-Nicolson scheme in time to discretize the model in (1) and (4). To solve AC equations with fully discretization, we can use central finite-difference discretization or a Fourier collocation method as described in [37]. With the periodic boundary condition, the numerical scheme can be written as: for any $n \geq 0$,

$$\frac{U_{n+1}^{i,j} - U_n^{i,j}}{\Delta t} = \epsilon^2 \Delta_h \frac{U_{n+1}^{i,j} + U_n^{i,j}}{2} + \frac{f(U_{n+1}^{i,j}) + f(U_n^{i,j})}{2} - g(U_n, U_{n+1}) \tag{6}$$

for $i, j = 1, 2, \cdots, N$, with

$$g(U_n, U_{n+1}) = \begin{cases} 0, & \text{for the classic AC,} \\ \frac{1}{N^2} \sum_{i',j'=1}^{N} \frac{f(U_{n+1}^{i',j'}) + f(U_n^{i',j'})}{2}, & \text{for the mass-conservative AC.} \end{cases}$$

This yields

$$U_{n+1}^{i,j} - \frac{\Delta t}{2}\epsilon^2 \Delta_h U_{n+1}^{i,j} = U_n^{i,j} + \frac{\Delta t}{2}\left(\epsilon^2 \Delta_h U_n^{i,j} + f(U_n^{i,j}) + f(U_{n+1}^{i,j}) - g(U_n, U_{n+1})\right), \tag{7}$$

for $i, j = 1, 2, \cdots, N$, where Δ_h denotes the five-point stencil for the Laplacian. It is known that the system (7) is guaranteed to have a unique solution U_{n+1} for any given U_n under a quite relaxed restriction on the time step size Δt and the resulting numerical solution has a second-order accuracy in both time and space. With picard iteration, the numerical scheme can be solved as: let $U_{n+1,0} = U_n$ and for $m = 0, 1, \cdots$, compute $U_{n+1,(m+1)}$ by solving

the linear system

$$\left(I - \frac{\Delta t}{2}\epsilon^2 \Delta_h\right) U^{i,j}_{n+1,(m+1)} = U^{i,j}_n + \frac{\Delta t}{2}\Big(\epsilon^2 \Delta_h U^{i,j}_n + f(U^{i,j}_n) + f(U^{i,j}_{n+1,(m)}) - g(U_n, U_{n+1,(m)})\Big), \tag{8}$$

for $i, j = 1, 2, \cdots, N$, and finally set $U_{n+1} = U_{n+1,m^*}$ if U_{n+1,m^*} satisfies certain convergence criteria.

The above system can be rewritten as

$$U_{n+1} = \mathcal{F}(t_n, t_{n+1}, U_n), \quad n \geq 0, \tag{9}$$

where $\mathcal{F}$ denotes the fully discrete operator that maps the input U_n to the output U_{n+1} through the relation (7) with the corresponding Δt. This operator $\mathcal{F}$ serves as the fine propagator in the Parareal algorithm.

3.2 Training the ACNN and mACNN Models

Building on the numerical scheme, we introduce the machine learning models employed as coarse propagators within the Parareal framework. Recall that the fine propagator $\mathcal{F}$, introduced in Section 3.1, denotes the nonlinear evolution operator defined by the fully discrete numerical scheme in equation (9). For a given state U_n, the operator $\mathcal{F}$ advances the solution to the next state U_{n+1}. In contrast, the nonlinear mapping $\mathcal{G}$ defined by the ACNN or mACNN model

$$\mathcal{G}(U_n; \Theta) : U_n \mapsto U_{n+1}, \tag{10}$$

where Θ represents the set of trainable parameters. The model $\mathcal{G}$ is trained so that

$$\mathcal{G}(U_n; \Theta) \approx \mathcal{F}(t_n, t_{n+1}, U_n), \tag{11}$$

thereby providing an approximation of the numerical evolution operator. Within the Parareal algorithm, $\mathcal{F}$ serves as the fine propagator, while $\mathcal{G}$ is used as the coarse propagator, and the interaction between these two operators forms the foundation of the proposed framework.

Fig. 1 illustrates the overall architecture of ACNN (top) and mACNN (bottom) developed in [11]. The input U_n is stored as a four-dimensional shape tensor $[1, 1, N, N]$, corresponding to the format `[batch, channel, height, width]`, and the output is the predicted state U_{n+1}. The network architecture comprises an input layer, several residual blocks (ResBlocks) [13], an output layer, and a boundary-limiter module. Importantly, both ACNN and mACNN avoid using pooling or fully connected layers. To respect the periodic boundary conditions inherent to the problem, all convolutional layers employ "circular" padding. Each convolutional layer uses 3×3 filters with a padding size of 1. The hyperbolic tangent (tanh) function serves as the activation function after each convolutional layer, except for the output layer. Following the input layer, the tensor shape becomes $[1, C, N, N]$, which is maintained in all intermediate layers. The final output tensor U_{n+1} has the shape $[1, 1, N, N]$ consistent with the input format. The bound limiter is used to enforce MBP, ensuring that the output remains within a prescribed interval $[\alpha, \beta]$. To solve the mass-conservative AC equation, an additional value is uniformly added to all entries of the network's output to enforce mass conservation before the bound-limited layer. Note that in the referenced article [11], the authors used a more complex architecture with three residual blocks and 16 channels (C=16) to achieve high-accuracy simulations. In contrast, our work emphasizes a trade-off between accuracy and computational efficiency, which is particularly important in the context of the

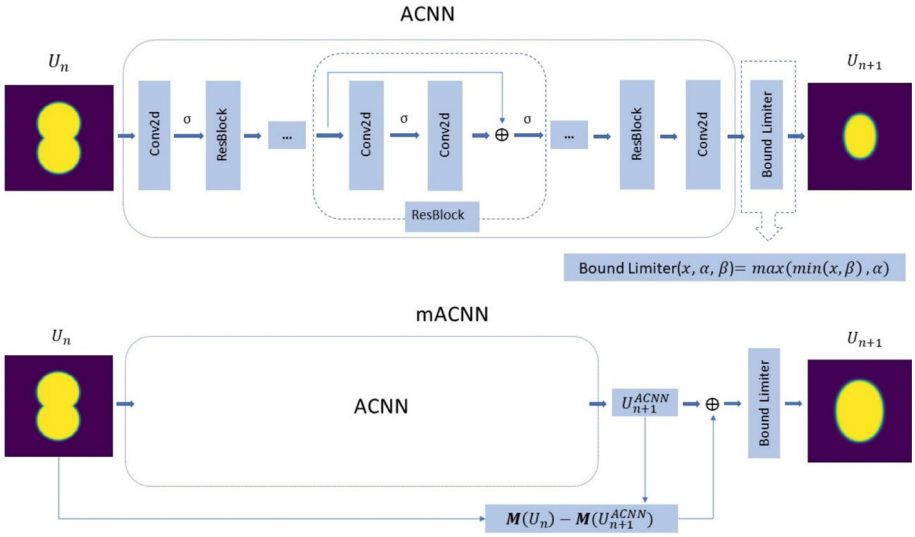


Fig. 1 [11] The structures of ACNN (top) and mACNN (bottom) which learn the dynamics of classical AC equation and mass-conservative AC equation respectively

Parareal algorithm. For the coarse propagator, it is crucial to maintain reasonable accuracy while ensuring fast inference to maximize parallel efficiency. Therefore, we adopt a lighter network with two residual blocks and 4 channels (C=4). Despite its reduced complexity, this model still delivers sufficiently accurate simulations, making it well-suited as a coarse propagator in the Parareal framework.

With the defined network architecture, the loss function can be defined as follows. Assuming R different initial conditions are trained simultaneously with batch size R, the loss function of the neural network model is defined using the fully discrete scheme (7) and measured in the L^2-norm:

$$\mathcal{L} = \frac{1}{RN^2} \sum_{r=1}^{R} \sum_{i,j=1}^{N} \Bigg[U_{n+1}^{i,j,(r)} - U_n^{i,j,(r)} - \frac{\Delta t}{2} \Big(\epsilon^2 \Delta_h (U_{n+1}^{i,j,(r)} + U_n^{i,j,(r)}) + f(U_{n+1}^{i,j,(r)}) + f(U_n^{i,j,(r)}) - g(U_{n+1}^{i,j,(r)}) - g(U_n^{i,j,(r)}) \Big) \Bigg]^2. \tag{12}$$

Because the loss function is derived directly from the fully discrete numerical scheme, the neural network model is consistent with the fine solver in the sense that it enforces the same discrete evolution law. This consistency ensures that the learned coarse propagator $\mathcal{G}$ remains faithful to the underlying discretization of the AC equations, making it particularly well-suited for integration into the Parareal algorithm, where alignment between fine and coarse solvers is critical for stability and convergence. Moreover, since the formulation involves the fully discrete representations of both U_n and U_{n+1}, the model can naturally accommodate nonlocal terms, thus extending its applicability to the mAC equation.

To train the ACNN and mACNN models following Algorithm 1, the training data are generated from multiple initial conditions, which are randomly partitioned into subsets and evolved over time. Training is carried out iteratively across time steps, with the network parameters updated using a loss function based on the discretization scheme (12). Consequently, the learned models are not restricted to a single trajectory, but are capable of generalizing across

arbitrary initial conditions. This property makes the neural network propagator particularly suitable for the Parareal algorithm, where the coarse solver can be applicable to a wide range of solution states.

We have introduced both the fine propagator $\mathcal{F}$, derived from the fully discrete numerical scheme, and the coarse propagator $\mathcal{G}$, defined by the neural network models ACNN and mACNN, whose loss functions are constructed directly from the fully discrete formulation. In the next section, we combine these two propagators within the Parareal framework and present the resulting time-parallel algorithm in detail.

Algorithm 1 Training strategy of ACNN/mACNN

Generate training data from problems with different initial conditions $\{U_0^{(r)}\}_{r=1}^{R}$.
Randomly partition the initial conditions into p subsets, each containing q samples ($R = pq$).
for each subset of initial conditions **do**
  Initialize U_0 with q initial condition samples.
  for each time step $n = 0, 1, \ldots, T_{\text{train}}/\Delta t$ **do**
    for $k = 1, \ldots, b$ **do** ▷ inner training loop
      Predict the next state: $U_{n+1} = \mathcal{G}(U_n; \Theta)$.
      Update network parameters Θ by minimizing the loss function (12).
    end for
    After b updates, fix U_{n+1} and use it as input for the next step.
  end for
end for
Repeat for multiple epochs until convergence.

3.3 Integrating the Trained ACNN and mACNN into the Parareal Algorithm

We now introduce the Parareal algorithm using ACNN and mACNN as the coarse propagator and analyze the speedup. For simplicity, we write $\mathcal{G}(U_n) = \mathcal{G}(U_n, \Theta) = U_{n+1}$. Algorithm 2 presents our proposed Parallel algorithm, and Fig. 2 illustrates the corresponding workflow. The algorithm begins with an initial coarse prediction

$$\mathbf{U}^0 = [U_0^0, U_1^0, \ldots, U_s^0],$$

where $U_0^0 = U_0$ is the prescribed initial condition, the coarse time step size is Δt and $U_{n+1}^0 = \mathcal{G}(U_n^0)$ for $n = 0, 1, \cdots, s-1$ with s denoting the total number of time steps. This initial trajectory $\mathbf{U}^0$ is then refined by the fine solver $\mathcal{F}$, which computes high-fidelity solutions in parallel as

$$U_{n+1,\mathcal{F}}^0 = \mathcal{F}(t_n, t_{n+1}, U_n^0), \qquad n = 0, 1, \ldots, s-1,$$

using a smaller fine time step Δt^*. This results in a sequence of intermediate states $[U_0^0, U_{1,\mathcal{F}}^0, \ldots, U_{s,\mathcal{F}}^0]$. In our implementation, the fine propagators $\mathcal{F}$ on different coarse time intervals $[t_n, t_{n+1}]$ for $n = 0, 1, \cdots, s-1$ are mutually independent and are therefore executed concurrently on multiple GPUs. When a sufficient number of GPUs is available, a one-to-one mapping between GPUs and coarse time intervals is used, so that all fine propagators can be executed fully in parallel within each Parareal iteration. When GPU resources are limited, multiple coarse time intervals are grouped and processed sequentially on the same GPU, while different groups are handled in parallel.

The updated solutions in the Parareal algorithm are computed iteratively using the formula:

$$U_{n+1}^k = \mathcal{G}(U_n^k) + \mathcal{F}(t_n, t_{n+1}, U_n^{k-1}) - \mathcal{G}(U_n^{k-1}), \tag{13}$$

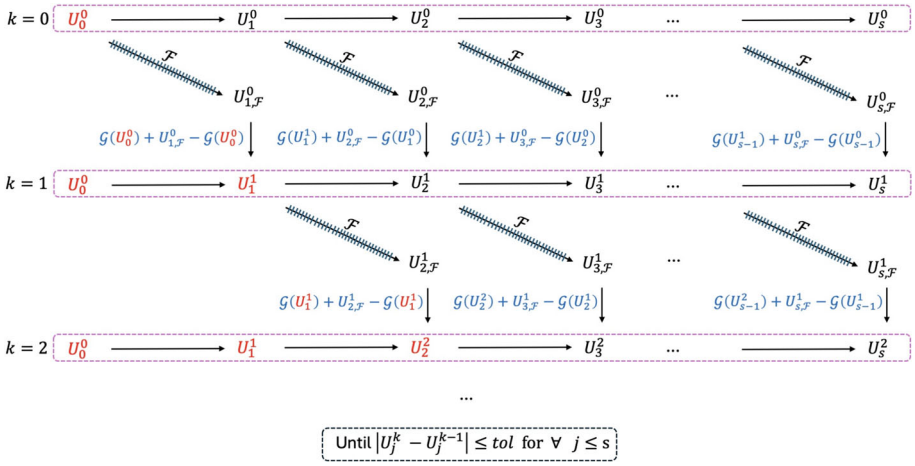


Fig. 2 Illustration of the Parareal algorithm. The coarse propagator here is labeled $\mathcal{G}$, whereas the fine propagator is labeled $\mathcal{F}$. The red font indicates that the solutions are already fine-solver simulations

until the sequence of iterates converges. Specifically, we declare convergence in iteration k if the following condition is satisfied $\forall\ j \le s, |U_j^k - U_j^{k-1}| < tol$ where tol is a user-defined tolerance threshold.

Since the initial condition U_0^0 is fixed, after the first iteration, we have

$$U_1^1 = \mathcal{G}(U_0^1) + \mathcal{F}(t_0, t_1, U_0^0) - \mathcal{G}(U_0^0) = \mathcal{F}(t_0, t_1, U_0^0), \tag{14}$$

where $U_0^1 = U_0^0 = U_0$. Thus, after the first update, U_1^1 exactly matches the solution obtained from the fine solver using the time step Δt^*. Consequently, after k iterations, the first k time-step solutions $[U_0^0, U_1^1, \cdots, U_k^k]$ will already match the output of the fine solver. This property implies that at most s Parareal iterations are sufficient to recover the fine solver solution. In practice, however, the objective of the Parareal method is to converge in far fewer iterations, i.e., $k \ll s$, in order to achieve a meaningful parallel speedup.

Algorithm 2 The Parareal Algorithm with ACNN/mACNN as the Coarse Solver

Require: Initial condition U^0, number of time steps s, tolerance tol
Generate initial guess $[U_0^0, U_1^0, \ldots, U_s^0]$ sequentially using the ACNN/mACNN model:
  $U_{n+1}^0 = \mathcal{G}(U_n^0)$ for $n = 0, 1, \ldots, s-1$
for $k = 1$ to s **do**
  Compute intermediate fine solutions in parallel using the fine solver:
    $U_{n+1,\mathcal{F}}^{k-1} = \mathcal{F}(t_n, t_{n+1}, U_n^{k-1})$ for $n = k-1, k, \ldots, s-1$ ▷ Using fine time step Δt^*
  for $n = k-1$ to $s-1$ **do**
    $U_{n+1}^k = \mathcal{G}(U_n^k) + U_{n+1,\mathcal{F}}^{k-1} - \mathcal{G}(U_n^{k-1})$ ▷ Update solution using Parareal correction
  end for
  if $|U_j^k - U_j^{k-1}| < tol$ for all $j \le s$ **then**
    Convergence achieved, exit loop.
  end if
end for
Ensure: Converged solution $[U_0^0, U_1^1, \cdots, U_k^k, \ldots, U_s^k]$

3.4 Discussion on the Computational Efficiency

The time complexity of the Parareal algorithm has been discussed in detail in [25]. We analyze the computational cost of the Parareal algorithm when a neural network-based coarse propagator is employed. Assume that the fine solver uses a time step Δt^* and the coarse solver advances the solution over a larger time step $\Delta t = c\,\Delta t^*$, where c is the time step ratio. Let t_{num} denote the average wall-clock time required to advance the solution by one fine time step size Δt^* using the numerical solver, to compute the solution over a time interval of length $s\Delta t = cs\Delta t^*$, a purely sequential fine solver requires a total cost of $T_{\text{num}} = cs\,t_{\text{num}}$. Let t_{nn} denote the average wall-clock time of one evaluation of the neural network coarse propagator. Assume that the Parareal algorithm terminates after k iterations and that the number of GPUs is sufficient to parallelize, then each iteration's parallel cost becomes ct_{num} and the total cost simplifies to:

$$\begin{aligned} T(k) &= ckt_{num} + (s + (s-1) + \cdots + (s-k))\,t_{nn} \\ &= ckt_{num} + \frac{(2s-k)(k+1)}{2} t_{nn}. \end{aligned}$$

Compared to the baseline cost $T_{num} = cst_{num}$, the Parereal approach is faster if:

$$\frac{(2s-k)(k+1)}{2} t_{nn} \le (s-k)ct_{num}.$$

The corresponding speedup is:

$$S_k = \frac{T_{num}}{T(k)} = \frac{1}{\frac{k}{s} + \frac{(2s-k)(k+1)}{2cs}\frac{t_{nn}}{t_{num}}} \le \min\left\{\frac{s}{k}, \frac{2cst_{num}}{(2s-k)(k+1)t_{nn}}\right\}. \tag{15}$$

The efficiency of the proposed Parareal framework with ACNN and mACNN as coarse propagators can be understood through three complementary observations. First, the use of a larger time-step ratio c is feasible because the neural coarse solver is not constrained by the stability restrictions that limit traditional numerical coarse solvers. The ACNN and mACNN models are trained with data generated at a temporal resolution, ensuring that the local dynamics are accurately represented. However, within the Parareal framework, the network can be applied recursively to advance the solution over a larger effective time step Δt by composing multiple evaluations of the trained model. In this way, the neural coarse solver remains stable and accurate, while enabling significantly more efficient coarse propagation than a numerical solver, whose stability often deteriorates when large time steps are used. Second, the number of iterations k required for convergence is directly related to the accuracy of the coarse solver: since ACNN and mACNN can achieve relatively higher accuracy, particularly in capturing sharp interfacial dynamics, fewer correction steps are needed, which further enhances efficiency. Third, the ratio $\frac{t_{\text{num}}}{t_{\text{nn}}}$ reflects the computational advantage of the neural coarse propagator, whose evaluation cost is much faster compared to the fine solver. Beyond efficiency, these models also offer strong generalization capabilities: they can be applied to arbitrary initial conditions without retraining and can naturally handle nonlocal terms in the mAC equation. Together, these properties ensure that ACNN and mACNN provide an accurate, efficient, and broadly applicable coarse propagator for the Parareal algorithm.

4 Numerical Experiments

In this section, we first illustrate a series of experiments for the classical AC equation (1) and the mass-conservative AC equation (4) in 2D and 3D on the convergence and accuracy of the Parareal algorithm. And then we perform the speedup on multiple GPUs comparing with fine solver time. Throughout all experiments, we fix the spatial domain $\Omega = [-0.5, 0.5]^d$ with $d = 2, 3$, the interfacial parameter $\epsilon = 0.01$ for 2D examples and $\epsilon = 0.02$ for 3D examples, and use $\Delta t = 0.1$ for the coarse function and $\Delta t^* = 0.001$ for the fine solver. To evaluate the accuracy of our Parareal algorithm, we calculate three types of errors under the relative L_2-norm

$$\mathcal{E}_{\mathcal{NN}} = \frac{\|\mathbf{U}_{\mathcal{NN}} - \mathbf{U}_{ref}\|_2}{\|\mathbf{U}_{ref}\|_2},$$

$$\mathcal{E}_{Para} = \frac{\|\mathbf{U}_{Para} - \mathbf{U}_{ref}\|_2}{\|\mathbf{U}_{ref}\|_2},$$

$$\mathcal{E}_{Num} = \frac{\|\mathbf{U}_{Num} - \mathbf{U}_{ref}\|_2}{\|\mathbf{U}_{ref}\|_2},$$

where $\mathbf{U}_{\mathcal{NN}}$ is the predicted solution generated by CNN models, $\mathbf{U}_{Para}$ is the updated simulation solution after the Parareal algorithm, $\mathbf{U}_{Num}$ is the fine numerical solution with Δt^* and $\mathbf{U}_{ref}$ is the reference solution. The reference solution is computed with the same spatial mesh and a sufficiently small time step size $\Delta t_{ref} = 0.0001$. We conducted the numerical experiments in PyTorch and all numerical experiments were performed on the Frontier supercomputer at Oak Ridge National Laboratory, which is equipped with AMD Instinct 250X GPUs with 64GB of memory.

4.1 Training Stability Across Models

To systematically assess the robustness and training stability of our proposed models, we performed ten independent training runs for each architecture of ACNN and mACNN bundle identical training data but with distinct random initializations. We then compute and visualize the mean predicted errors along with the mean $\pm$ one standard deviation bands over time in both 2D (Fig. 3) and 3D (Fig. 4) settings. These results help quantify the variability due to stochastic training effects and provide insight into the consistency of the learned dynamics.

As illustrated in Fig. 3, the 2D simulations show that both ACNN (left) and mACNN (right) produce highly stable and consistent predictions across all runs. The narrow error bands reflect low variance, indicating that the models reliably converge despite randomness in initialization or optimizer trajectories. For the 3D case shown in Fig. 4, the models exhibit slightly increased variability, as expected due to the higher dimensional complexity of the problem. However, even in this setting, the standard deviation remains modest and the mean prediction error remains well below 0.035 throughout the entire time horizon. These results reinforce the reliability and generalization capabilities of our architectures.

Given the observed stability, we proceed with reporting detailed results based on a single representative model per architecture for the remainder of our experiments.

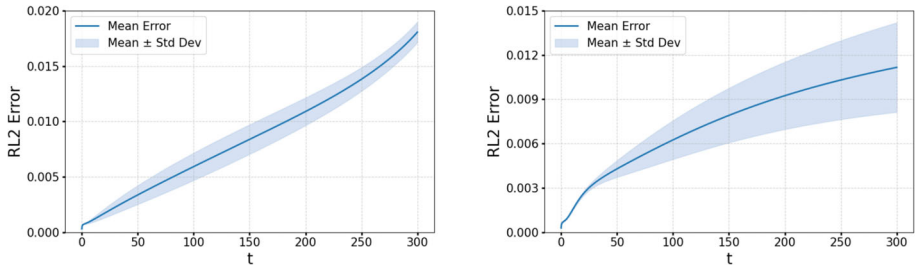


Fig. 3 2D CNN models: Mean predicted errors and one-standard-deviation bands across ten independently trained models over time. The left panel shows results for ACNN, while the right panel presents results for mACNN

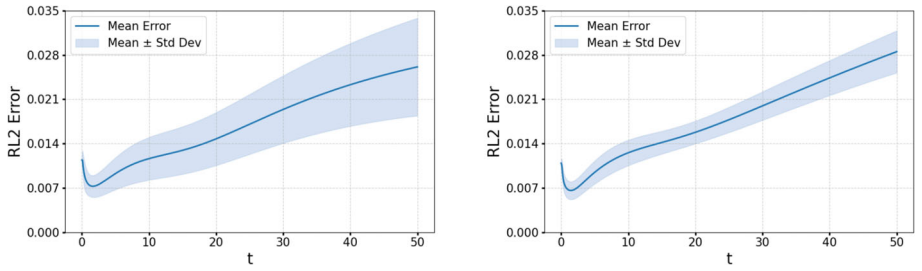


Fig. 4 3D CNN models: Mean predicted errors and one-standard-deviation bands across ten independently trained models over time. The left panel corresponds to ACNN, and the right panel to mACNN

4.2 Dynamics Prediction for 2D Examples

Next, we present 2D benchmark examples to demonstrate the effectiveness of the Parareal algorithm in terms of both convergence and accuracy for the dynamics of the classical and mass conservative AC equations. Two well-known benchmark examples are considered: merging bubbles and grain coarsening. For spatial discretization, we set the mesh size to $h = 1/256$.

4.2.1 Merging Bubbles

We consider the bubble merging problem, which is governed by the initial condition as follows:

$$u_0(x, y) = \max\left(\tanh\left(\frac{0.2 - \sqrt{(x-0.14)^2 + y^2}}{\epsilon}\right), \tanh\left(\frac{0.2 - \sqrt{(x+0.14)^2 + y^2}}{\epsilon}\right)\right). \tag{16}$$

We predict the solution up to $T = 300$, resulting in a total of $s = 3000$ time steps. Figs. 5 and 6 illustrate the convergence behavior of the Parareal algorithm with respect to the iteration index k, and the corresponding prediction errors after the k-th iteration. The results show that the difference in the supremum norm $\|U^k - U^{k-1}\|_\infty$ decreases rapidly, reaching 10^{-6} around 10 iterations. With each Parareal iteration k, the solution becomes increasingly close to the fine solver. After a sufficient number of iterations (e.g., $k = 10$), the error becomes negligible, showing convergence. Figs. 7 and 8 plot the predicted solution after 10 iterations of the Parareal algorithm and the associated errors at times $t = 10, 50, 200, 300$ for the classical

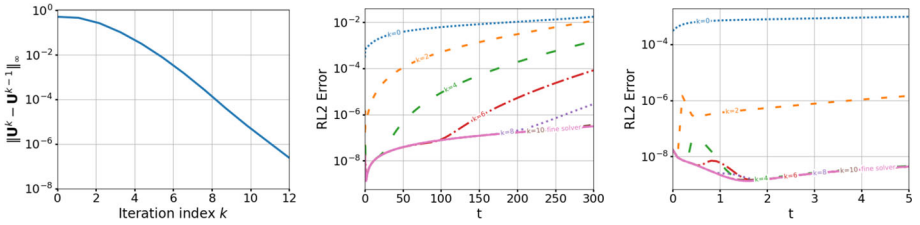


Fig. 5 Classical AC in 2D: $\|U^k - U^{k-1}\|_\infty$ along the iteration index k (left), the corresponding relative errors after the $k = 0, 2, 4, 6, 8$ and 10 iteration (middle) and a zoom-in of the error curves at early times (right). Note $k = 0$ means the coarse propagator errors without Parareal algorithm

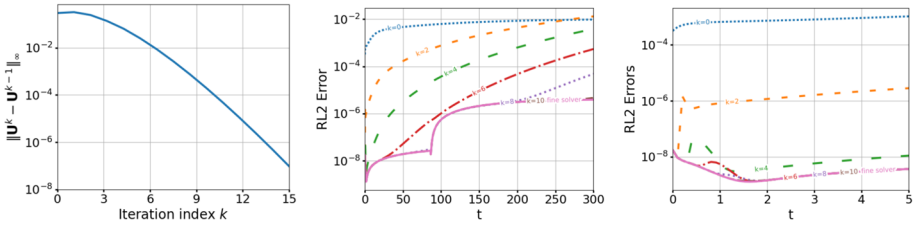


Fig. 6 Mass-conservative AC in 2D: $\|U^k - U^{k-1}\|_\infty$ along the iteration index k (left), the corresponding relative errors after the $k = 0, 2, 4, 6, 8$ and 10 iteration (middle) and a zoom-in of the error curves at early times (right). Note $k = 0$ means the coarse propagator errors without Parareal algorithm

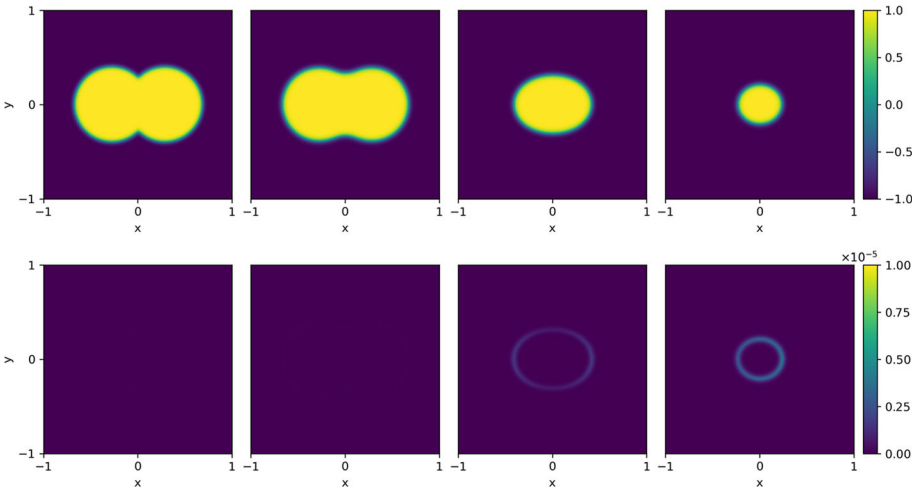


Fig. 7 Classical AC in 2D: The predicted solutions (top row) after 10 iterations of the Parareal algorithm and the corresponding numerical errors (bottom row) at $t = 10, 50, 200, 300$

and mass-conservative AC equations, respectively. The observed prediction errors are mainly localized in the interfacial regions between phases, and the maximum error compared to the reference is around 0.0001.

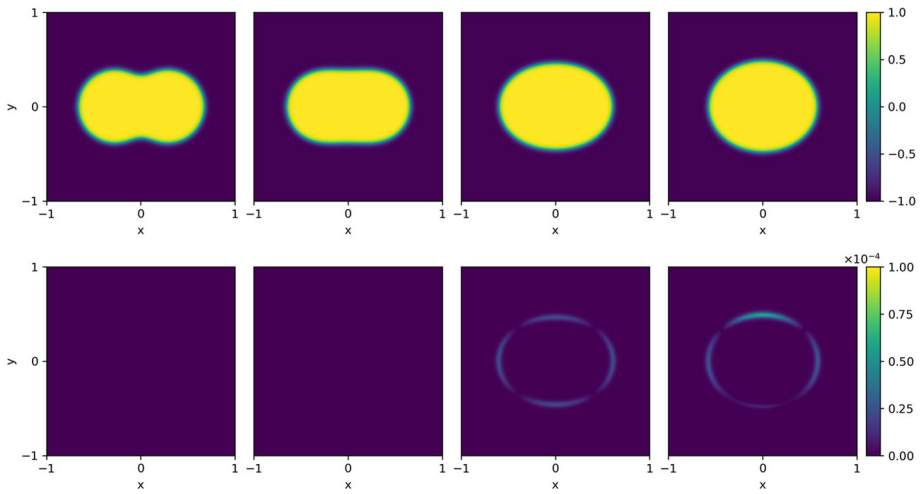


Fig. 8 Mass-Conservative AC in 2D: The predicted solutions (top row) after 10 iterations of the Parareal algorithm and the corresponding numerical errors (bottom row) at $t = 10, 50, 200, 300$

4.2.2 Grain Coarsening

Next, we consider a random initialized condition of the form $u_0(x, y) = 0.9\mathbf{rand}(\cdot)$, and test it on classical and mass-conservative AC equations. For the case of grain coarsening, we predict the solution up to $T = 500$, resulting in a total of $s = 5000$ time steps. Figs. 9 and 10 (left) show the convergence behavior for both cases. Compared to the merging bubbles example, the convergence is slower; however, the same accuracy trend is observed in Figs. 9 and 10 (middle), where the predicted solutions achieve nearly the same accuracy as the fine solver around 10 iterations. In Figs. 9 and 10 (right), we zoom in on the error evolution over a short initial time interval. In the zoomed-in Figs, the sharp jump in the error is a structural feature of the Parareal algorithm. After the k-th Parareal iteration, the solution coincides with the fine solver on the first k coarse time intervals, leading to zero error for $t \leq k\Delta t$. In addition, the early stage is dominated by rapid interface motion and phase merging, during which small corrections can significantly reduce error. After the interfaces have merged, the solution evolves more smoothly and the error gradually stabilizes. As time progresses further, the Parareal algorithm continues to propagate the solution sequentially in time, and residual errors from earlier intervals accumulate, leading to an overall increase of the error at later times.

In Figs. 11 and 12, we plot the Parareal predicted solutions after 10 iterations and their corresponding error maps at times $t = 0.5, 10, 200, 500$ for the classical and mass-conservative AC equations, respectively. During the grain coarsening process, the classical model exhibits gradual shrinkage and eventual disappearance of one phase (the yellow region), while the mass-conservative model preserves the overall volume of the phase, with no phase completely disappearing. Compared with reference solutions, the maximum predicted error is around 0.005, and the errors occur only in the transition between the two phases.

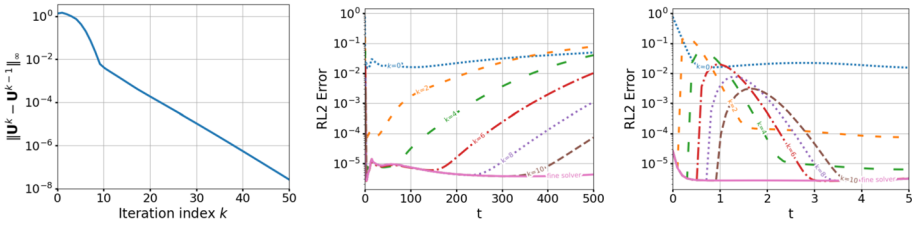


Fig. 9 Classical AC in 2D: $\|U^k - U^{k-1}\|_\infty$ along the iteration index k (left), the corresponding relative errors after the $k = 0, 2, 4, 6, 8$ and 10 iteration (middle) and a zoom-in of the error curves at early times (right). Note $k = 0$ means the coarse propagator errors without Parareal algorithm

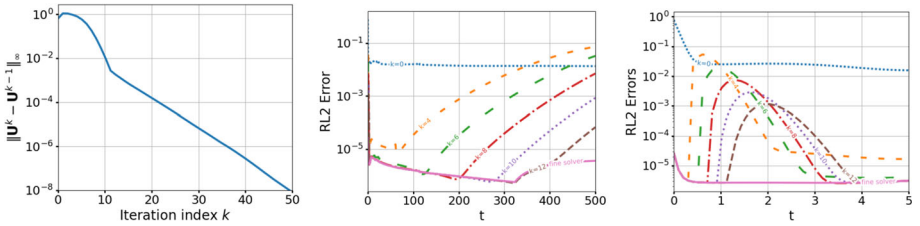


Fig. 10 Mass-conservative AC in 2D: $\|U^k - U^{k-1}\|_\infty$ along the iteration index k (left), the corresponding relative errors after the $k = 0, 4, 6, 8, 10, 12$ and 10 iteration (middle) and a zoom-in of the error curves at early times (right). Note $k = 0$ means the coarse propagator errors without Parareal algorithm

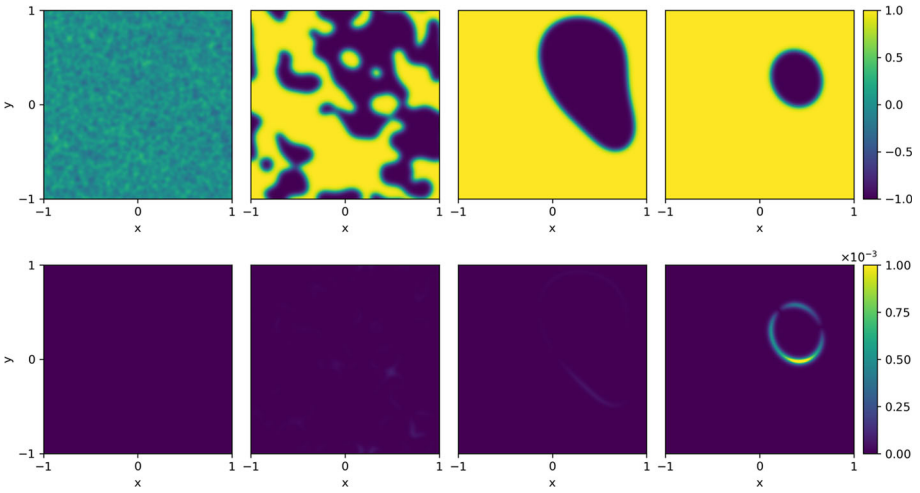


Fig. 11 Classical AC in 2D: The predicted solutions (top row) after 10 iterations of the Parareal algorithm and the corresponding numerical errors (bottom row) at $t = 0.5, 10, 200, 500$

4.2.3 Impact of the Coarse Propagator on Parareal Convergence

In this section, we investigate the impact of the coarse propagator on Parareal convergence by comparing a numerical coarse solver based on the Crank-Nicolson scheme with a learned CNN-based coarse propagator. To assess Parareal convergence using a numerical coarse propagator, we consider the 2D classical AC merging bubbles example and integrate up to

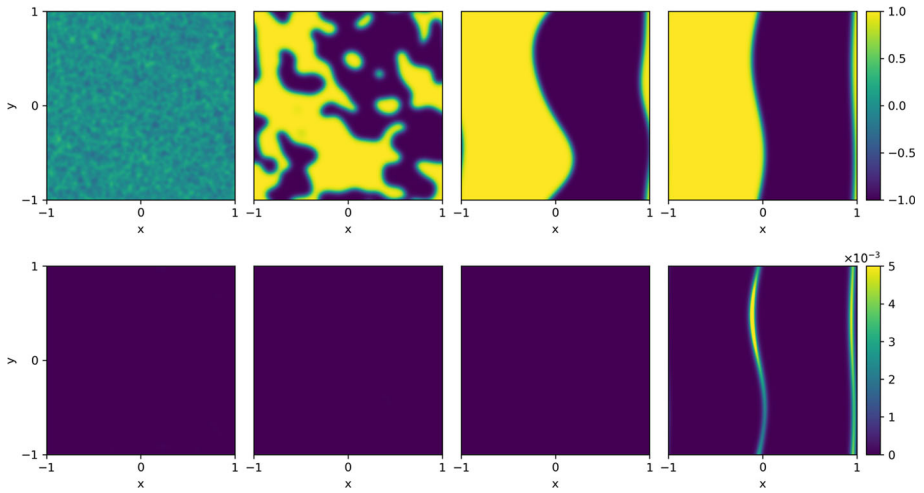


Fig. 12 Mass-Conservative AC in 2D: The predicted solutions (top row) after 10 iterations of the Parareal algorithm and the corresponding numerical errors (bottom row) at $t = 0.5, 10, 200, 500$

$T = 10$ in Fig. 13. When CNN is used as the coarse propagator (left panel of Fig. 13), Parareal converges rapidly, reaching errors on the order of 10^{-8} within approximately $k = 4$ to 6 iterations. By contrast, when the Crank-Nicolson numerical solver is used as the coarse propagator with the same time step $\Delta t = 0.1$ (right panel of Fig. 13), Parareal iterations exhibit very slow or stagnating convergence, despite the Crank-Nicolson scheme providing a relatively accurate initial guess at $k = 0$.

From the experiments, we observe that the Parareal correction step in the equation (13), modifies the solution at each iteration and may distort the structure of solution, especially the MBP. This effect is more obvious for random initial conditions, where the maximum norm of the updated solution may grow and even blow up during the iterations. As a result, the Parareal iterations could fail to converge when a numerical solver is used as the coarse propagator. In contrast, for the CNN-based coarse propagator, the CNN incorporates a maximum-bound layer, which ensures that the updated solutions remain within the admissible bounds. In terms of computational cost, the reported approximation time corresponds to a single coarse time step from t_n to t_{n+1} with coarse time step size Δt. For the CNN-based coarse propagator, the forward prediction from t_n to t_{n+1} requires approximately 0.0011 s per coarse step. By contrast, the CN numerical coarse solver requires about 0.0068 s per coarse step under stable conditions. Moreover, when used within the Parareal framework, the correction step may affect the nonlinear Picard iterations in the numerical solver, especially for random initial conditions. In some cases, the Picard solver reaches the prescribed maximum iteration number without convergence, significantly increasing the computational cost. Therefore, a direct comparison of total wall-clock time is not entirely meaningful, since the numerical coarse solver may become substantially slower due to instability in the Parareal iterations. Consequently, the CNN-based coarse propagator leads to faster and more robust Parareal convergence compared to the numerical solver.

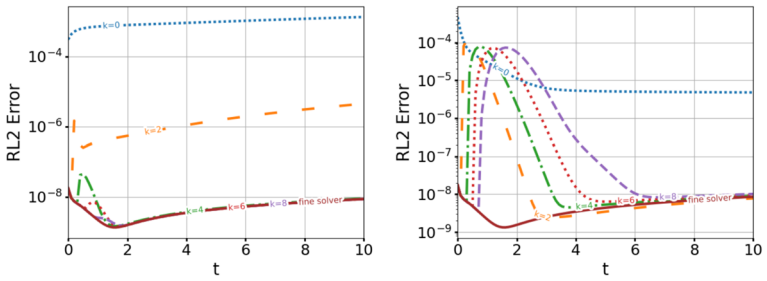


Fig. 13 2D AC merging bubbles example: Parareal convergence using (left) a CNN-based coarse propagator with $\Delta t = 0.1$ and (right) a numerical solver coarse propagator with $\Delta t = 0.1$

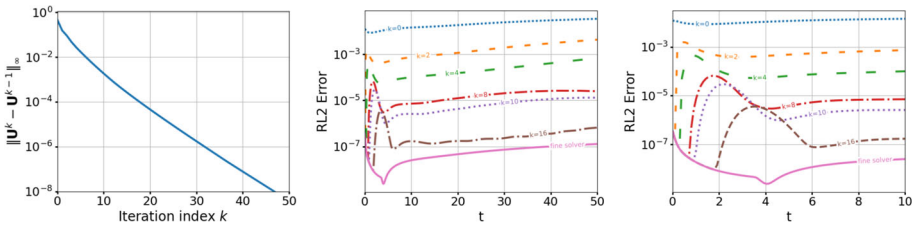


Fig. 14 Classical AC in 3D: $\|U^k - U^{k-1}\|_\infty$ along the iteration index k (left), the corresponding relative errors after the $k = 0, 2, 4, 8, 10$ and 16 iteration (middle) and a zoom-in of the error curves at early times (right). Note $k = 0$ means the coarse propagator errors without the Parareal algorithm

4.3 Dynamics Prediction for 3D Examples

In this section, we present 3D benchmark examples to demonstrate the effectiveness of the Parareal algorithm in terms of both convergence and accuracy for the dynamics of AC equations. For spatial discretization, we set the mesh size to $h = 1/128$.

4.3.1 Star Shape

We consider the "star" shape as the initial configuration and predict the solution up to $T = 50$ for the classical AC equation. Similarly to the 2D problem, we first test the convergence and accuracy of the Parareal algorithm on 3D models. Figs. 14 and 15 show the convergence and accuracy behavior for both cases. We observe that convergence is fast, with the supremum norm difference $\|U^k - U^{k-1}\|_\infty$ reaching 10^{-8} after approximately 50 iterations. As the number of iterations increases, the accuracy improves and gradually approaches the solution obtained by the fine solver. Although convergence is slower than in the 2D cases, the predicted error reduces to around 10^{-5} after 10 iterations. Figs. 16 and 17 illustrate the predicted solution after 10 iterations of the Parareal algorithm and the associated errors at times $t = 1, 10, 20, 50$ for the classical and $t = 1, 10, 20, 30$ mass-conservative AC equations, respectively. For the classical AC model, we observe that the initial star-shaped structure gradually smooths into a ball, which then progressively shrinks over time. In contrast, under the conservative AC setting, the ball eventually evolves into a stable, well-shaped structure with the same volume as the initial state, reflecting the mass conservation property inherent in the conservative formulation. The maximum predicted error is around 0.0005 compared to the reference solutions.

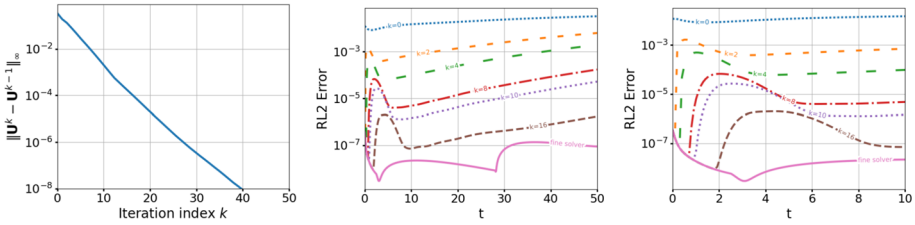


Fig. 15 Mass-Conservative in 3D: $\|U^k - U^{k-1}\|_\infty$ along the iteration index k (left), the corresponding relative errors after the $k = 0, 2, 4, 6, 10$ and 16 iteration (middle) and a zoom-in of the error curves at early times (right). Note $k = 0$ means the coarse propagator errors without the Parareal algorithm

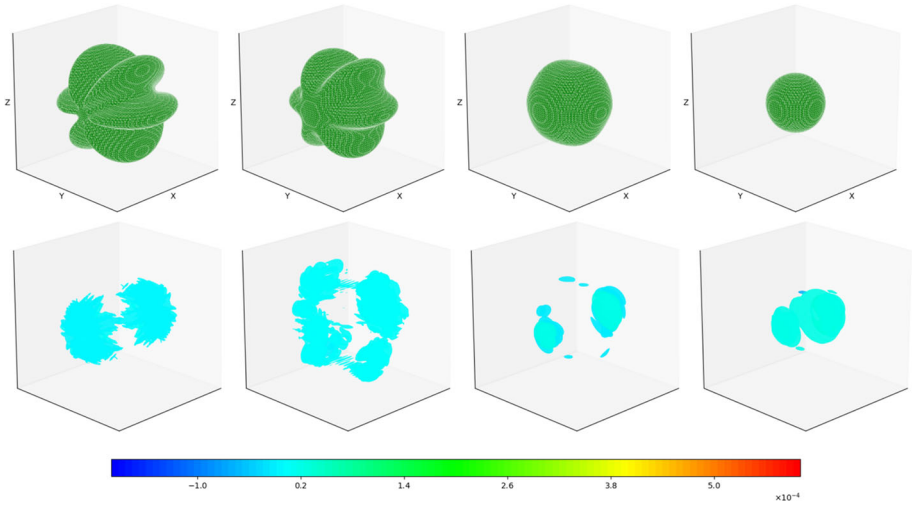


Fig. 16 Classical AC in 3D: The predicted solutions (top row) after 10 iterations of the Parareal algorithm and the corresponding numerical errors (bottom row) at $t = 1, 10, 20, 50$

4.3.2 Grain Coarsening

Finally, we apply the Parareal algorithm to predict the 3D grain coarsening dynamics with an initial condition generated randomly by $u_0(x, y, z) = 0.9\,\mathbf{rand}(\cdot)$. The simulation time interval is set to $[0, 50]$ for both the classical and conservative AC equations. The convergence and precision with the iteration index k are shown in Figs. 18 and 19. The convergence and accuracy of the predicted solutions exhibit behavior similar to that observed in the previous example. Figs. 20 and 21 illustrate the predicted results with 10 iterations of the Parareal algorithm and associated errors at times $t = 1, 10, 20, 50$ for the classical and conservative AC equations, respectively. Compared to the reference solution, the maximum errors are approximately 0.002.

4.4 Discussions on Speedup

In this part, we consider estimating the speedup obtained by applying the Parareal algorithm. To this end, we consider random initial conditions with ending time $T = 500$ for 2D cases and ending time $T = 50$ for 3D cases. Since the accuracy depends on the number of Parareal

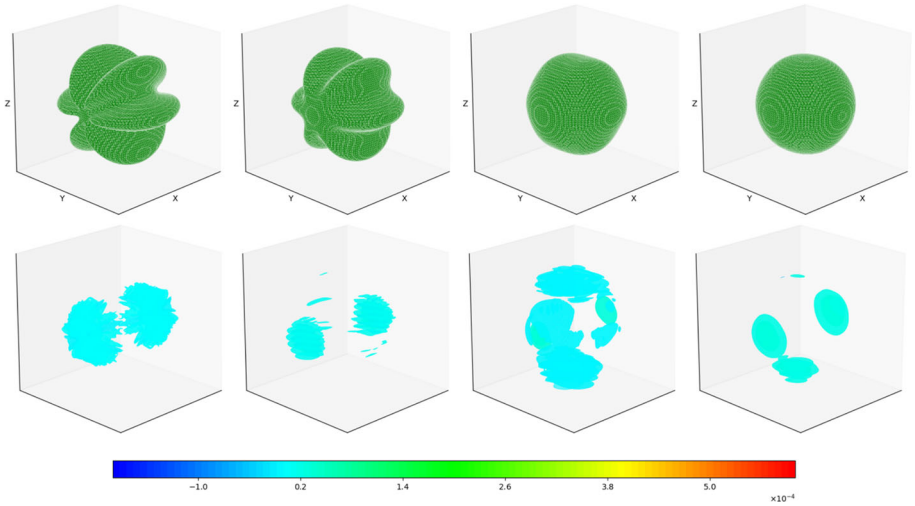


Fig. 17 Mass-Conservative AC in 3D: The predicted solutions (top row) after 10 iterations of the Parareal algorithm and the corresponding numerical errors (bottom row) at $t = 1, 10, 20, 30$

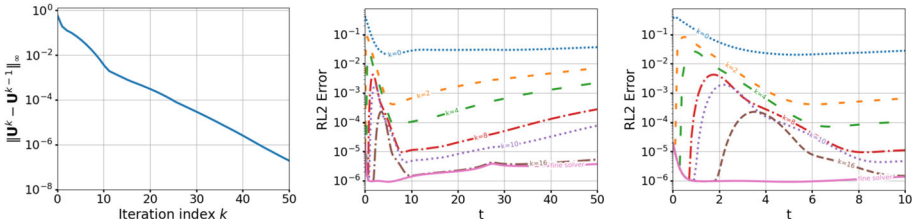


Fig. 18 Classical AC in 3D: $\|U^k - U^{k-1}\|_\infty$ along the iteration index k (left), the corresponding relative errors after the $k = 0, 2, 4, 8, 10$ and 16 iteration (middle) and a zoom-in of the error curves at early times (right). Note $k = 0$ means the coarse propagator errors without the Parareal algorithm

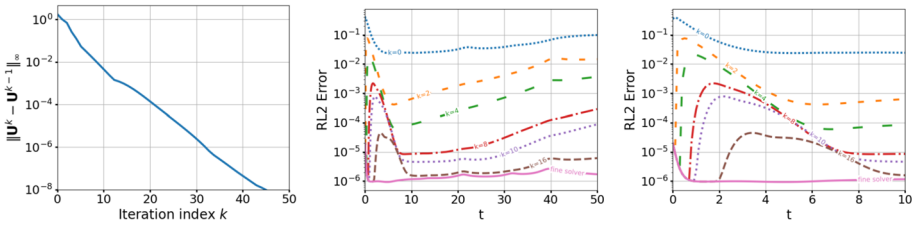


Fig. 19 Mass-Conservative AC in 3D: $\|U^k - U^{k-1}\|_\infty$ along the iteration index k (left), the corresponding relative errors after the $k = 0, 2, 4, 8, 10$ and 16 iteration (middle) and a zoom-in of the error curves at early times (right). Note $k = 0$ means the coarse propagator errors without the Parareal algorithm.

iterations, we present the accuracy and the running times across multiple iteration numbers and GPU counts, and compare them to the fine solver. Figs. 22 and 24 show the accuracy for different numbers of iterations relative to the fine solver. In Figs. 23 and 25, the running times are shown. Colored lines correspond to different numbers of GPUs (8, 16, 32, 64, and 128), while the black horizontal line denotes the fine solver as a baseline. The goal of the Parareal algorithm is to significantly reduce simulation time while maintaining accuracy comparable

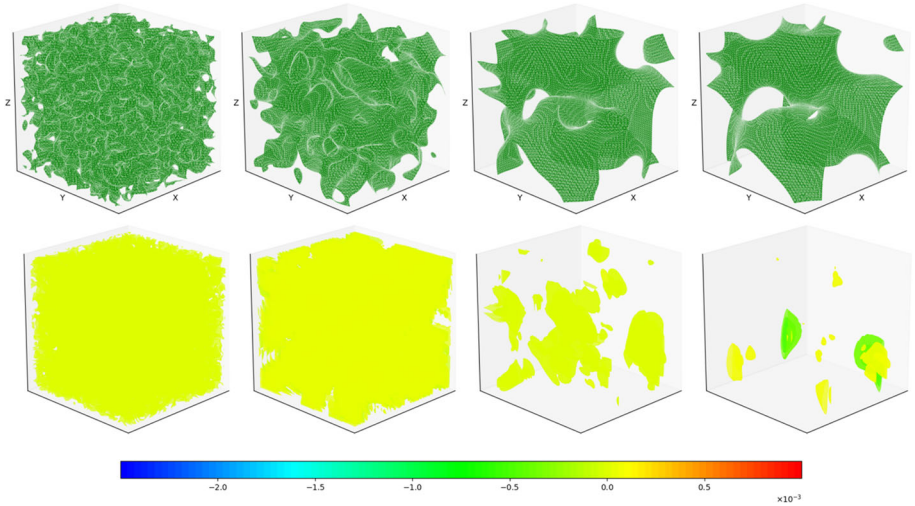


Fig. 20 Classical AC in 3D: The predicted solutions (top row) after 10 iterations of the Parareal algorithm and the corresponding numerical errors (bottom row) at $t = 1, 10, 20, 50$

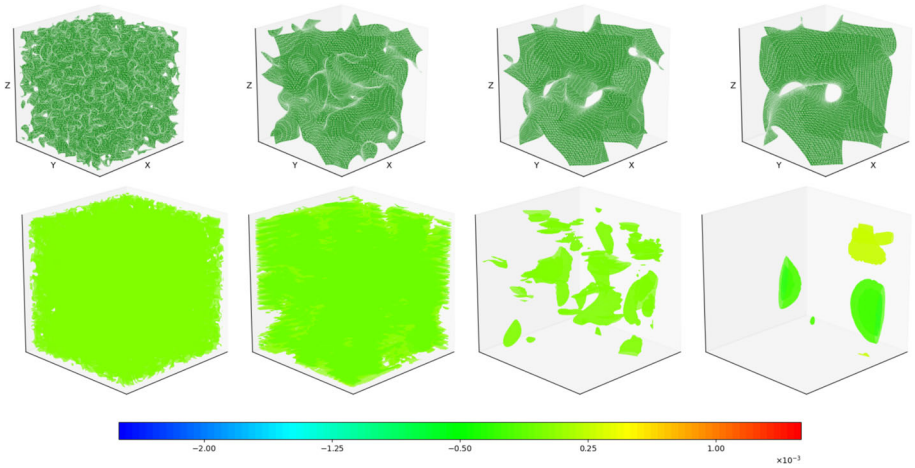


Fig. 21 Mass-Conservative AC in 3D: The predicted solutions (top row) after 10 iterations of the Parareal algorithm and the corresponding numerical errors (bottom row) at $t = 1, 10, 20, 50$

to that of the fine solver. As expected, increasing the number of GPUs generally reduces the computation time.

For the 2D problems, we first present the solution error evolution with respect to the Parareal iterations in Fig. 22. The Parareal algorithm achieves high accuracy in around 10 iterations, closely matching the fine solver. At this point, the running time in Fig. 23 is significantly reduced by approximately a factor of six for both the classical and mass-conservative AC equations when a sufficient number of GPUs is used. This demonstrates the effectiveness of the algorithm in accelerating time-dependent simulations without sacrificing accuracy. However, the benefit of adding more GPUs begins to diminish as the communication overhead increases.

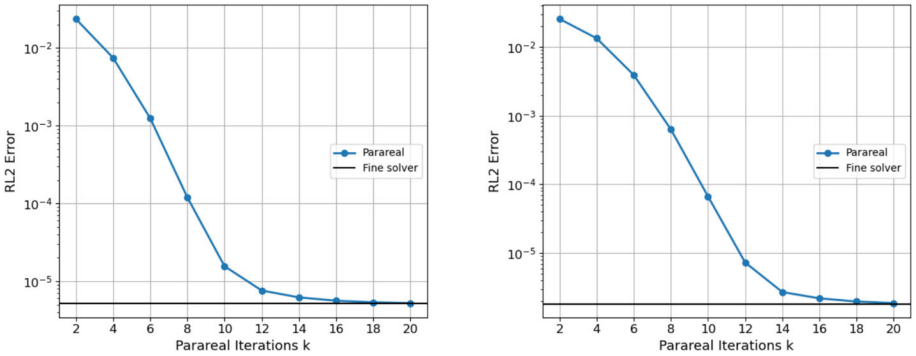


Fig. 22 2D AC: The Parareal solution errors with different number of iterations for classical AC (left) and mAC (right) equations

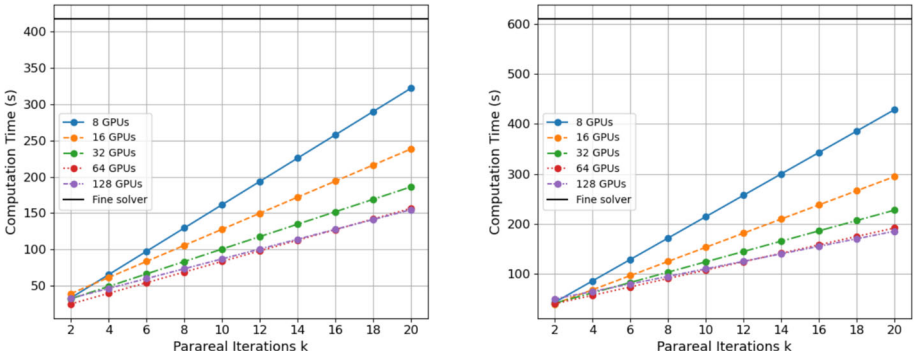


Fig. 23 2D AC: The Parareal running times with different number of iterations on different number of GPUs for classical AC (left) and mass-conservative AC (right) equations

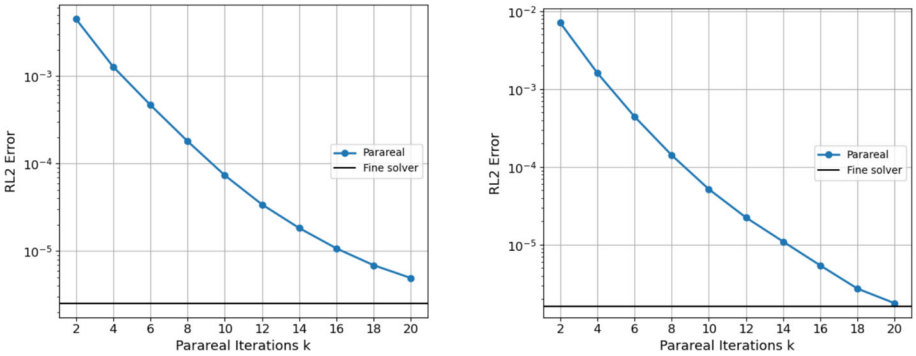


Fig. 24 3D AC: The Parareal solution error with different number of iterations for classical AC (left) and mass-conservative AC (right) equations

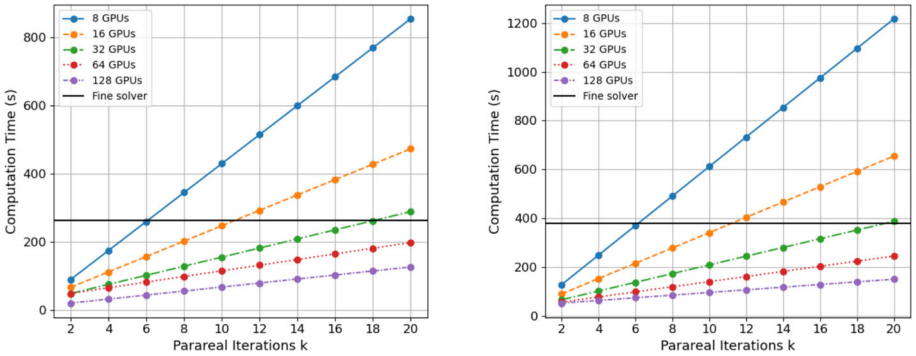


Fig. 25 3D AC: The Parareal running times with different number of iterations on different number of GPUs for classical AC (left) and mass-conservative AC (right) equations

In the 3D experiments, Fig. 24 illustrates the solution error evolution with respect to the Parareal iterations. The error decreases with iterations and converges to the fine solver accuracy after approximately 20 iterations. Fig. 25 shows the computational time as a function of iterations for different numbers of GPUs. The Parareal algorithm still offers computational advantages, particularly when using a large number of GPUs (e.g., 64 or 128). However, the speedup is less pronounced compared to the 2D case, primarily because the simulation is only performed up to $T = 50$ using 500 time steps, which limits the overall parallel time horizon. By this time, the solution has already reached a steady state and remains essentially unchanged. In such shorter time integration scenarios, the overhead of inter-GPU communication can outweigh the benefits of parallel execution. However, the Parareal method still achieves around a two to three times speedup around 20 iterations for both 3D variants of the AC equation when using sufficient GPUs.

Overall, the results show that with a well-trained coarse propagator and adequate hardware resources, the Parareal algorithm can achieve substantial acceleration in both 2D and 3D settings while preserving the accuracy of the fine solver across various iteration counts and GPU configurations.

5 Conclusions

This work demonstrates the effectiveness of using CNNs as the coarse propagator in the Parareal method for accelerating the simulation of both classical and mass-conservative AC equations. Numerical results in both 2D and 3D settings show that the proposed framework achieves significant speedup while maintaining accuracy comparable to the fine solver, especially when a sufficient number of GPUs are employed. The Parareal iterations converge rapidly, and in the 2D case, accuracy is attained within ten iterations, achieving up to 2$\sim$6 times runtime reduction. Although the speedup in 3D is less pronounced due to a shorter simulation horizon, the results still validate the scalability and robustness of the approach.

Looking forward, this method has strong potential for applications in complex multi-scale systems such as weather forecasting, where a fast AI model can serve as the coarse propagator, while the physical model, being too slow to run frequently, acts as the fine correction step. Unlike traditional Parareal settings that require a physics-based coarse solver, our approach removes this requirement by fully replacing the coarse phase solver with a trained

neural network. This direction offers a promising path to real-time forecasting in scientific computing domains that currently face strict runtime limitations. On the other hand, this framework relies on the availability of a suitable learning-based coarse propagator. In the present work, this component is constructed using convolutional neural networks, which are naturally defined on structured grids. As a result, the current implementation is restricted to problems on structured domains. However, the framework itself is not inherently limited to a specific model architecture. As long as an appropriate learning-based coarse solver is available, it can be seamlessly integrated into the Parareal framework. Therefore, extending the approach to irregular domains depends on the development or adoption of suitable models, such as operator learning methods or graph-based neural networks, that are compatible with unstructured data. This will be a direction for future work.

Acknowledgements This work is supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research, Applied Mathematics program, under the contracts ERKJ388 and ERKJ443. ORNL is operated by UT-Battelle, LLC., for the U.S. Department of Energy under Contract DE-AC05-00OR22725. L. Ju acknowledges the support from the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research program under grants DE-SC0025527. E. C. Cyr acknowledges support through funding from the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research at Sandia National Laboratories. Sandia National Laboratories is a multimission laboratory managed and operated by National Technology & Engineering Solutions of Sandia, LLC, a wholly owned subsidiary of Honeywell International Inc., for the U.S. Department of Energy's National Nuclear Security Administration under contract DE-NA0003525. This paper describes objective technical results and analysis. Any subjective views or opinions that might be expressed in the paper do not necessarily represent the views of the U.S. Department of Energy or the United States Government.

Data Availability Data available on request from the authors.

Declarations

Conflict of Interest This manuscript has been authored by UT-Battelle, LLC, under contract DE-AC05-00OR22725 with the US Department of Energy (DOE). The US government retains and the publisher, by accepting the article for publication, acknowledges that the US government retains a nonexclusive, paid-up, irrevocable, worldwide license to publish or reproduce the published form of this manuscript, or allow others to do so, for US government purposes. DOE will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan.

References

1. Agboh, W., Grainger, O., Ruprecht, D., Dogar, M.: Parareal with a learned coarse model for robotic manipulation. Comput. Vis. Sci. **23**(1), 8 (2020)
2. Allen, S.M., Cahn, J.W.: A microscopic theory for antiphase boundary motion and its application to antiphase domain coarsening. Acta Metall. **27**(6), 1085–1095 (1979)
3. Beneš, M., Chalupeckỳ, V., Mikula, K.: Geometrical image segmentation by the Allen-Cahn equation. Appl. Numer. Math. **51**(2–3), 187–205 (2004)
4. Chen, C., Yang, Y., Xiang, Y., Hao, W.: Learn sharp interface solution by homotopy dynamics. arXiv e-prints arXiv:2502 (2025)
5. Doan, C.-K., Hoang, T.-T.-P., Ju, L.: Fully discrete error analysis of first-order low regularity integrators for the Allen-Cahn equation. Numer. Methods for Partial Differ. Equ. **39**(5), 3594–3608 (2023)
6. Doan, C.-K., Hoang, T.-T.-P., Ju, L., Schratz, K.: Low regularity integrators for semilinear parabolic equations with maximum bound principles. BIT Numer. Math. **63**, 2 (2023)
7. Du, Q., Ju, L., Li, X., Qiao, Z.: Maximum bound principles for a class of semilinear parabolic equations and exponential time-differencing schemes. SIAM Rev. **63**(2), 317–359 (2021)
8. Falgout, R.D., Friedhoff, S., Kolev, T.V., MacLachlan, S.P., Schroder, J.B.: Parallel time integration with multigrid. SIAM J. Sci. Comput. **36**(6), C635–C661 (2014)

9. Feng, X., Prohl, A.: Numerical analysis of the Allen-Cahn equation and approximation for mean curvature flows. Numer. Math. **94**, 33–65 (2003)
10. Gander, M.J., Wu, S.-L., Zhou, T.: Time parallelization for hyperbolic and parabolic problems. Acta Numer **34**, 385–489 (2025)
11. Geng, Y., Teng, Y., Wang, Z., Ju, L.: A deep learning method for the dynamics of classic and conservative Allen-Cahn equations based on fully-discrete operators. J. Comput. Phys. **496**, 112589 (2024)
12. Gunther, S., Ruthotto, L., Schroder, J.B., Cyr, E.C., Gauger, N.R.: Layer-parallel training of deep residual neural networks. SIAM J. on Math. of Data Sci. **2**(1), 1–23 (2020)
13. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 770–778 (2016)
14. Huang, J., You, R., Zhou, T.: Frequency-adaptive multi-scale deep neural networks. Comput. Methods Appl. Mech. Eng. **437**, 117751 (2025)
15. Ibrahim, A.Q., Götschel, S., Ruprecht, D.: Parareal with a physics-informed neural network as coarse propagator. In: European Conference on Parallel Processing, pp. 649–663. Springer (2023)
16. Kirby, A., Samsi, S., Jones, M., Reuther, A., Kepner, J., Gadepally, V.: Layer-parallel training with gpu concurrency of deep residual neural networks via nonlinear multigrid. In: 2020 IEEE High Performance Extreme Computing Conference (HPEC), pp. 1–7. IEEE (2020)
17. Krishnapriyan, A., Gholami, A., Zhe, S., Kirby, R., Mahoney, M.W.: Characterizing possible failure modes in physics-informed neural networks. Adv. Neural. Inf. Process. Syst. **34**, 26548–26560 (2021)
18. Lee, Y., Park, J., Lee, C.-O.: Parareal neural networks emulating a parallel-in-time algorithm. IEEE Trans. on Neural Netw. and Learning Syst. **35**(5), 6353–6364 (2022)
19. Li, J., Ju, L., Cai, Y., Feng, X.: Unconditionally maximum bound principle preserving linear schemes for the conservative Allen-Cahn equation with nonlocal constraint. J. Sci. Comput. **87**, 1–32 (2021)
20. Lions, J.-L., Maday, Y., Turinici, G.: Résolution d'edp par un schéma en temps «pararéel». Comptes Rendus de l'Académie des Sci.-Ser. I-Math. **332**(7), 661–668 (2001)
21. Liu, X., Xu, B., Cao, S., Zhang, L.: Mitigating spectral bias for the multiscale operator learning. J. Comput. Phys. **506**, 112944 (2024)
22. Lorin, E.: Derivation and analysis of parallel-in-time neural ordinary differential equations. Ann. Math. Artif. Intell. **88**(10), 1035–1059 (2020)
23. Mattey, R., Ghosh, S.: A novel sequential method to train physics informed neural networks for Allen-Cahn and Cahn-Hilliard equations. Comput. Methods Appl. Mech. Eng. **390**, 114474 (2022)
24. Meng, X., Li, Z., Zhang, D., Karniadakis, G.E.: PPINN: Parareal physics-informed neural network for time-dependent PDEs. Comput. Methods Appl. Mech. Eng. **370**, 113250 (2020)
25. Minion, M.: A hybrid parareal spectral deferred corrections method. Commun. in Appl. Math. and Comput. Sci. **5**(2), 265–301 (2011)
26. Nguyen, H., Tsai, R.: Numerical wave propagation aided by deep learning. J. Comput. Phys. **475**, 111828 (2023)
27. Raissi, M., Perdikaris, P., Karniadakis, G.E.: Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. J. Comput. Phys. **378**, 686–707 (2019)
28. Rubinstein, J., Sternberg, P.: Nonlocal reaction–diffusion equations and nucleation. IMA J. Appl. Math. **48**(3), 249–264 (1992)
29. Shao, D., Rappel, W.-J., Levine, H.: Computational model for cell morphodynamics. Phys. Rev. Lett. **105**(10), 108104 (2010)
30. Shen, J., Yang, X.: Numerical approximations of Allen-Cahn and Cahn-Hilliard equations. Discrete Contin. Dynam. Systems **28**(4), 1669–1691 (2010)
31. Tang, T., Yang, J.: Implicit-explicit scheme for the Allen-Cahn equation preserves the maximum principle. J. Comput. Math. 451–461 (2016)
32. Wang, H., Qian, X., Sun, Y., Song, S.: A modified physics informed neural networks for solving the partial differential equation with conservation laws. Available at SSRN 4274376 (2022)
33. Wheeler, A.A., Boettinger, W.J., McFadden, G.B.: Phase-field model for isothermal phase transitions in binary alloys. Phys. Rev. A **45**(10), 7424 (1992)
34. Xu, J., Li, Y., Wu, S., Bousquet, A.: On the stability and accuracy of partially and fully implicit schemes for phase field modeling. Comput. Methods Appl. Mech. Eng. **345**, 826–853 (2019)
35. Xu, Z.-Q.J., Zhang, Y., Luo, T.: Overview frequency principle/spectral bias in deep learning. Commun. on Appl. Math. and Comput. **7**(3), 827–864 (2025)
36. Yalla, G.R., Engquist, B.: Parallel in time algorithms for multiscale dynamical systems using interpolation and neural networks. Inst. for Comput. Eng. and Sci. (2018)
37. Yang, J., Du, Q., Zhang, W.: Uniform l^p-bound of the Allen-Cahn equation and its numerical discretization. Inter. J. of Numer. Anal. & Model. **15**, 213–227 (2018)

38. Zhao, C.L.: Solving Allen-Cahn and Cahn-Hilliard equations using the adaptive physics informed neural networks. Commun. in Comput. Phys. **29**, 3 (2020)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.